

FaaSRep: Efficient Strict Serializability for Serverless Workflows via Workflow Reconciliation

Anonymous Author(s)

Abstract

Serverless computing, represented by Function-as-a-Service (FaaS), is increasingly adopted for deploying complex applications via stateful workflows. However, the lack of strong consistency (i.e., strict serializability) transaction support hinders its broader applicability. Existing solutions are inadequate, either offering only weak consistency or suffering severe performance degradation from concurrency control models that blindly retry requests after conflicts.

We therefore present FaaSRep, a high-performance serverless framework that provides strict serializability through *workflow reconciliation*, which resolves conflicts by learning dependencies from an optimistic execution and using them to guide an organized retry. FaaSRep builds dependencies at validation time, using the validator’s global access view without extra execution-path communication. It further uses a hybrid optimistic/pessimistic protocol to preserve correctness when application-level aborts invalidate the learned order. To reduce the cost of reconciliation, FaaSRep re-executes only affected functions and uses placement information from the initial execution to establish a peer-to-peer fast path during retry. Compared with state-of-the-art systems on real-world applications, FaaSRep improves average throughput by 1.46–2.73× over Concord and 3.67–9.01× over Beldi.

1 Introduction

Serverless computing, also known as *Function-as-a-Service* (FaaS), has gained widespread adoption in the cloud-native era thanks to its advantages such as ease of use and pay-as-you-go billing [11, 31, 45]. To support complex applications like data analytics pipelines [15, 16, 21] and machine learning inference chains [14, 20], modern serverless platforms have evolved from executing isolated functions to orchestrating multi-step workflows. They commonly represent a workflow as a directed acyclic graph (DAG), where nodes are functions and edges capture invocation order and control dependencies [34, 38, 41]. This model is increasingly used not only for stateless pipelines, but also for applications whose functions access and update shared external state, such as orders, inventories, and payments [59, 62].

Such stateful workflows often require transaction semantics to ensure correctness [59, 62]. For example, an online shopping workflow [9, 10] may need to check inventory, deduct stock, and create an order. These state operations should either all take effect or none should. Meanwhile, concurrent requests should not lead to anomalies such as over-selling. We refer to such workflows as *transactional serverless*

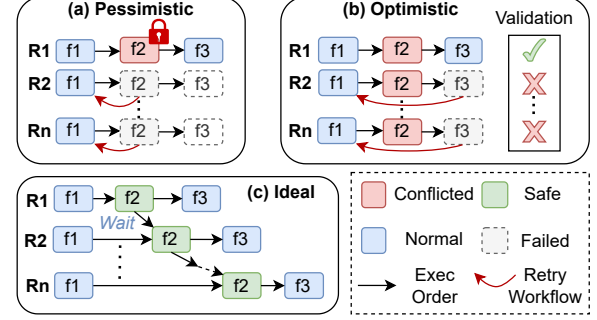


Figure 1. Comparison of conflict handling methods. Pessimistic/optimistic approaches retry failed requests repeatedly, while the ideal way organizes conflicting functions.

workflows: multiple functions within one workflow jointly form a transaction with strict serializability [19]. Their effects commit or abort atomically, and strict serializability ensures that concurrent workflow requests is equivalent to a real-time-respecting serial execution.

Existing serverless transaction systems mainly enforce these guarantees through pessimistic or optimistic concurrency control. Figure 1 compares them using a simple example: concurrent requests $R_1, R_2, \dots, R_n$ execute the same three-function workflow $f_1 \rightarrow f_2 \rightarrow f_3$. Their f_2 functions update the same shared object and therefore conflict. *Pessimistic* approaches coordinate shared-state accesses during execution. For example, Beldi [59] uses remote locks, whereas Concord [50] employs a cache-coherence protocol to prevent concurrent updates to external state by conflicting requests. *Optimistic* approaches [33] allow requests to execute first and validate their read/write sets later. After the conflict is detected, these approaches handle a failed attempt in the same way: the conflicted workflow request is discarded and retried from scratch.

This conflict-handling strategy is inefficient because retry does not change how concurrent requests interact. After a failed attempt, the retried request re-enters the system and competes for the same shared objects again. If the contention persists, the request may fail again and trigger another retry. Thus, existing systems can spend much of their time in an unbounded cycle of conflict, abort, and retry. Under high contention, this blind competition can dominate end-to-end latency. In our experiments, retry overhead increases latency by up to 4× (§3.2).

The key observation of this paper is that a failed serverless workflow attempt is not only expensive, but also informative.

In the example above, the failed attempt can serve as an information-discovery phase. Once the runtime observes that f_2 in these requests conflicts, it can turn this conflict into an explicit ordering constraint for the next attempt, for example requiring $R_k.f_2$ to run after $R_{k-1}.f_2$. The retry therefore no longer repeats competition, but follows the order learned from the first attempt.

We call this conflict-handling model *workflow reconciliation*. It separates conflict discovery from conflict resolution, as shown in Figure 1 (c). The first execution runs optimistically to reveal which functions from concurrent workflow requests access the same objects. Requests commit normally if no conflict is found. Otherwise, the runtime selects a serialization order and constructs a dependency graph that orders conflicting function instances. The retry follows this graph, serializing conflicting accesses through explicit constraints while allowing independent functions to proceed concurrently. Workflow reconciliation thus turns repeated abort-and-retry cycles into an information-guided retry.

However, harmonizing workflow reconciliation with serverless efficiently is non-trivial. First, dependency construction requires a global view of function-level accesses across concurrent requests, but functions may be placed independently across worker nodes [38, 41]. A naive solution must either query multiple nodes or maintain a centralized access log, introducing coordination overhead on the execution path. Second, application-level aborts [59] can invalidate dependencies inferred from the initial execution. For example, if $R_1.f_2$ in Figure 1 aborts, continuing $R_2.f_2$ under the same dependency order may therefore make it read from an invalid predecessor and then break the serialization order. Third, reconciliation would lose much benefit if retries still pay the same control-plane scheduling cost [35, 38] and re-execute the entire workflow. In Figure 1, only f_2 is involved in the conflict, but a naive retry also reruns unaffected functions such as f_1 and f_3 , wasting computation and scheduling work.

In this paper, we present *FaaSRep*, a transactional runtime that realizes workflow reconciliation for stateful serverless workflows to provide strict serializability. FaaSRep addresses the above serverless-specific challenges with three techniques. First, it constructs the reconciliation dependency graph at validation time. By piggybacking on stale-read checking step, FaaSRep obtains a global view of concurrent accesses without introducing a separate cross-node graph-construction phase. Second, it uses a hybrid optimistic/pessimistic reconciliation protocol: the common case remains optimistic, while requests affected by application-level aborts switch to a conservative mode when the inferred order becomes unstable. Third, it performs function-level re-execution with a peer-to-peer fast path. During the retry phase of reconciliation, FaaSRep re-executes only dirty functions and necessary dependents, skips clean functions whose results remain valid, and uses placement information from

the initial run to let containers communicate directly, reducing repeated control-plane scheduling.

We evaluate FaaSRep on an 8-node cluster. Using real-world applications, FaaSRep improves average throughput by $1.46\text{--}2.73\times$ over Concord and $3.67\text{--}9.01\times$ over Beldi. This paper makes the following contributions:

- We identify *workflow reconciliation* as a conflict-handling model for transactional serverless workflows. It uses information revealed by one optimistic execution to convert observed conflicts into dependency-constrained retry, instead of relying on unbounded blind retries.
- We design a strict-serializable reconciliation protocol for serverless workflows. The protocol constructs function-level dependencies by piggybacking on validation and handles application-level aborts with a hybrid optimistic/pessimistic mode.
- We implement FaaSRep, a transactional serverless runtime that supports function-level re-execution and a peer-to-peer fast path, reducing redundant computation and repeated control-plane scheduling during the retry phase.

2 Related Work

Transactional support in serverless computing with strong consistency. Beldi [59] implements a two-phase locking (2PL) protocol to ensure strict serializability, maintaining locks in a global storage service. For efficiency, Concord [50] utilizes a directory-based cache coherence protocol, while Boki [30] and Halfmoon [44] record lock entries in a shared log abstraction. However, as Boki and Halfmoon periodically update log indices by subscribing to the storage layer, they only provide *snapshot isolation* [18], which prevents concurrency problems like phantom reads but is not serializable. Another work, RTSFaaS [62], exploits a lease-based concurrency control protocol, but requires a priori knowledge of the entire access set to build its task-precedence graph.

Serverless transactions with weak consistency. A series of works turns to weak consistency guarantees to prevent the overhead caused by maintaining strong consistency [40, 48, 49, 56, 60]. Among them, AFT [48] deploys a fault-tolerance shim between the computing and storage layers, offering *Read Atomicity* [25] guarantees. CloudBurst [49] and FaaSSTCC [40] offer *Transactional Causal Consistency* guarantees by checking whether the causal dependency is violated during runtime. All these approaches cannot prevent concurrent write conflicts, thereby violating serializability.

Transaction support in the traditional database domain. Some works focus on transactions on a single database node [22, 29, 37, 54]. To improve the 2PL parallelism, Bamboo [29] allows transactions to release locks early. For distributed databases [26, 51–53], Calvin [52] avoids 2PC overhead by pre-ordering transactions before execution, limiting concurrency. Besides, a line of work [24, 26, 57] reduces

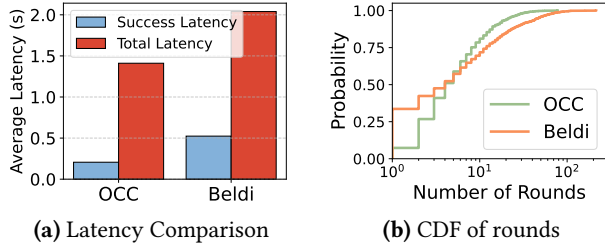


Figure 2. Execution results on Beldi and OCC. Many requests require repeated execution rounds.

abort cost by transaction healing and repair. They obtain data dependencies through static analysis or local execution-time tracking. The transactions in these works consist solely of data operations and will not proactively abort, which is incompatible with serverless functions.

3 Background and Motivation

3.1 Transactional Serverless Workflow

Applications in serverless computing are often structured as workflows of functions and represented as directed acyclic graphs (DAGs). A workflow triggers downstream functions through the control plane [34]. Because serverless platforms create function instances on demand and place them independently across worker nodes [11, 31, 36], a single workflow request may span multiple nodes [34, 38].

This paper focuses on transactional serverless workflows such as financial systems [6]. Their functions access shared objects through data APIs, and multiple functions in the same request jointly form a strict serializable transaction [1, 19, 27]. Strict serializability requires concurrent requests to produce the same effects as a serial execution that respects real time: if one request commits before another begins, the former must appear earlier in the serial order.

Unlike pure database transactions, serverless workflows contain application logic. A workflow request may also abort because of application-level conditions, such as insufficient inventory or balance [3, 27, 59]. We refer to such aborts as application-level aborts, to distinguish them from conflict-induced aborts triggered by concurrency control.

3.2 Implications of Existing Solutions

Existing serverless transaction systems mainly follow either pessimistic or optimistic concurrency control. Pessimistic systems, such as Beldi [59], Boki [30], and Concord [50], coordinate shared-state accesses during execution using locks or coherence protocols. Optimistic concurrency control (OCC) [12, 33, 54] allows requests to execute first, records their read/write sets, and then validates them to detect conflicts. It can also validate requests in batches to reduce network overhead [53].

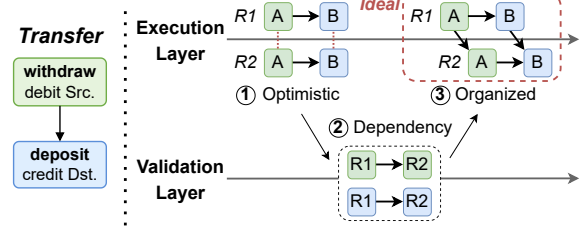


Figure 3. The execution diagram of workflow reconciliation.

These designs differ in where they detect conflicts, but they converge after a conflict is found: the failed attempt is discarded, and the workflow request is retried again, as illustrated by the red arrow in Figure 1. To quantify this cost, we run a travel reservation workload [1] in a closed-loop setting. Figure 2a separates the latency of the final successful attempt from the total end-to-end latency. In Beldi and OCC, the final successful attempt accounts for only 14.6% and 25.7% of the total latency, respectively. Figure 2b further shows that about 40% of requests require more than ten attempts before completion. Thus, the dominant cost is not the execution that commits, but the sequence of failed attempts.

The root cause is that these systems simply re-execute the workflow and again compete with concurrent requests for the same shared objects. If contention persists, the request can fail and retry repeatedly. The limitation also affects the Two-Phase Locking (2PL) protocol used by Beldi and Boki, because deadlock prevention schemes such as Wait-Die [46] still force lower-priority transactions to abort under lock contention.

3.3 Opportunity of Workflow Reconciliation

3.3.1 Ideal Way: Organized Execution. An ideal conflict-handling model should not let concurrent requests passively compete for shared objects. Instead, it should organize conflicting accesses before execution. If the system knows which requests access the same objects, it can plan their access order in advance: one request updates the object first, and the other observes the state produced by its predecessor. This avoids the repeated retry cycle of existing systems, because conflicts are resolved by explicit ordering instead of by reacting to failed attempts afterward.

The red-boxed part of Figure 3 illustrates this ideal process with a transfer workflow. The workflow has two functions: *withdraw* debits the source account, and *deposit* credits the destination account. Suppose two concurrent requests, R_1 and R_2 , transfer money between the same accounts A and B . If the system already knows this access pattern and chooses the serial order $R_1 \prec R_2$, it can execute R_2 .*withdraw* after R_1 .*withdraw* and R_2 .*deposit* after R_1 .*deposit*. The two requests then complete without racing on the same accounts, while unrelated accesses remain concurrent.

The key difficulty is that this order requires access information before execution. In serverless workflows, however, functions access external state through data APIs inside their function bodies, so their concrete read/write sets are hard to obtain in advance.

3.3.2 Workflow Reconciliation Methodology. Our key observation is that failed executions discarded by existing systems contain the information needed to approximate ideal organized execution. During a failed attempt, the runtime can observe which functions access external transactional objects and collect each function’s read/write set. Thus, a failed execution can act as an information-discovery phase rather than pure wasted work.

Therefore, we introduce *workflow reconciliation*, a two-stage protocol that separates information discovery from ordered execution. In the first stage, concurrent workflow requests execute optimistically and produce function-level metadata. In the second stage, FaaSRep selects a serialization order and uses the observed metadata to construct a dependency graph, through which conflicts are resolved by an organized retry.

Execution flow. Figure 3 illustrates the execution flow of workflow reconciliation. R_1 and R_2 first execute without coordination and expose their accesses to accounts A and B . The validation layer observes that the two requests conflict on the same accounts, chooses the order $R_1 \prec R_2$, and constructs dependencies from R_1 to R_2 at the corresponding functions. During the retry, R_2 waits only where it depends on R_1 , so the conflicting account accesses are serialized without repeating an unbounded abort-and-retry cycle.

3.4 Challenges and Solutions

However, designing a workflow reconciliation-based scheme for serverless environments is nontrivial.

3.4.1 Cross-Node Dependency Construction. Building the dependency graph requires a global view of function-level read/write metadata. In serverless platforms, this metadata is naturally distributed across worker nodes because function invocations are scheduled independently and one workflow may span multiple machines. A naive design can collect metadata from workers on demand or maintain a centralized access log during execution, but both add remote communication and shared coordination to the execution path. As a result, they incur overhead similar to remote locking in Beldi [59], where remote communication accounts for more than 25% of total latency (Section 8.7).

Solution: Delayed dependency construction. FaaSRep constructs dependencies at OCC-style validation time, where the validator has a global view of data accesses across concurrent requests. By piggybacking dependency construction on the existing stale-read checking step, FaaSRep obtains the graph with minimal overhead.

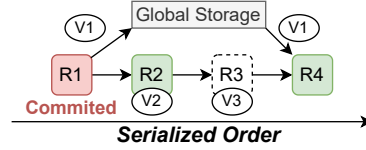


Figure 4. Application-level aborts can invalidate an inferred dependency order. R_1 – R_4 are serialized by the dependency graph. R_1 writes data version V_1 to the storage. R_2 is not committed yet, and buffers the version V_2 locally.

3.4.2 Application-level aborts Can Invalidate Inferred Orders. The application-level aborts triggered by serverless functions can make a previously valid dependency graph unsafe. Figure 4 shows an example. If R_3 aborts, R_4 loses the predecessor information provided by that edge. As a result, R_4 can only read the older committed version V_1 from global storage. This causes R_4 to miss an update V_2 that should appear before it in the serialized order, thereby violating strict serializability. Rebuilding the graph after each abort is also unsafe as a general strategy, because later application-level aborts may trigger further invalidations and lead to cascading, unbounded restart [29].

Solution: Hybrid optimistic/pessimistic mode. FaaSRep introduces a pessimistic mode to handle application-level aborts. In this mode, a request is retried only after its necessary dependency has been resolved, and will not suffer another re-execution. To balance correctness and performance, requests first attempt an optimistic retry and fall back to pessimistic mode only when necessary.

3.4.3 Avoid redundant work during retry. Even with a correct dependency graph, reconciliation can lose its advantage if retry follows the standard serverless path. Serverless control planes route function invocations and schedule successors after predecessors complete, adding non-negligible indirect communication overhead that can reach 181.3 ms for real-world applications [42]. If each retry step still goes through the control plane, reconciliation repeatedly pays this scheduling and routing cost. Moreover, conflicts usually affect only part of a workflow. Retrying the entire workflow, as in previous systems [50, 59], repeats functions whose outputs remain valid, wasting computation and masking the function-level structure exposed by serverless workflows.

Solution: Function-level re-execution and fast path. During the retry phase, FaaSRep marks functions that read expired data as dirty, and then propagates this status along workflow dependencies. A function is clean only if its own reads remain unchanged and all predecessors are also clean; otherwise, it becomes dirty. Thus, FaaSRep re-executes only dirty functions and reuses the previous results of clean ones. It further uses placement information from the initial execution to establish direct container-to-container communication, avoiding repeated control-plane routing.

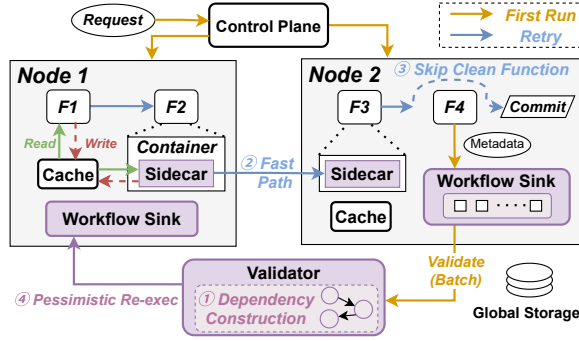


Figure 5. FaaSRep architecture. The workflow $f_1 \rightarrow f_2 \rightarrow f_3 \rightarrow f_4$ spans multiple worker nodes. Each worker maintains a local cache for external transactional state, whose entries may be stale with respect to the versions in global storage.

4 Overview of FaaSRep

Figure 5 shows the architecture of FaaSRep, a transactional runtime for serverless workflows that realizes workflow reconciliation to provide strict serializability.

The initial execution of FaaSRep leverages the common setting of optimistic protocols [12, 33]. The control plane schedules workflow functions across worker nodes, while each function accesses external transactional states through a lightweight sidecar that provides `read(key)`, `write(key, value)`, and `abort()` APIs. The sidecar reads from the local cache, buffers writes until commit, and records function-level metadata, including read/write sets and container placement. As the workflow proceeds, this metadata propagates along the workflow DAG and is collected by the local workflow sink. When a request finishes, the sink batches the complete metadata of multiple requests and submits it to the validator.

During validation, besides checking stale accesses, the FaaSRep validator also constructs dependencies for the batch at the same time (① & §5.1). After validation, the sidecar organizes workflow retries at function granularity. It bypasses the control plane through the inter-container fast path (② & §6.3) and re-executes only dirty functions (③ & §6.2). Each dirty function reads the correct data version as specified by its upstream dependencies in the graph. If a request aborts during retry, FaaSRep validator and the workflow sink cooperate to convert the affected successors to pessimistic mode and re-execute them (④ & §5.2).

Scope. FaaSRep mainly targets transactional workflows with stable access sets. These workflows re-execute the same transactional logic under identical input, and they access external states solely through explicit data APIs, without requiring prior knowledge of the access set. For workflows that violate this property, FaaSRep treats them as application-level aborts. Section 7.3 discusses this in detail.

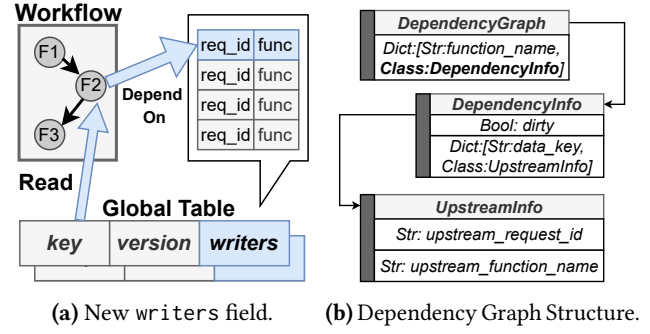


Figure 6. Validator-side dependency construction and the dependency graph representation.

5 Workflow Reconciliation Protocol

In this section, we first describe how FaaSRep constructs dependencies with low overhead during validation. We then explain how the hybrid optimistic/pessimistic mode works. Finally, we prove the correctness of the protocol.

5.1 Delayed Dependency Construction

The first step of workflow reconciliation is to construct a dependency graph. As discussed in Section 3.4.1, however, a straightforward method would introduce communication overhead. FaaSRep therefore delays dependency construction to validation time, and reuses the access metadata already collected for batch validation.

In standard OCC [12, 26], the validator maintains a *global table* that records the latest version of each data key. During validation, the validator processes requests in the order in which they are added to the validation batch, which defines the *equivalent serial order* $\prec_B$ for each batch B : if request R_i appears before request R_j in B , then R_i precedes R_j in the serial history represented by this validation result. For each request, OCC checks its read set against the global table. A stale read causes the request to be discarded, while a valid request commits and updates the table with its write set.

FaaSRep preserves this structure, but extends each entry in the global table with a `writers` field recording uncommitted writers of the key, as shown in Figure 6a. These writers may come from earlier batches. For each read, FaaSRep first checks the corresponding `writers` field. If it is non-empty, FaaSRep adds a dependency from the latest writer of this key to the reading function and marks the reader as *dirty*. Otherwise, FaaSRep performs the standard OCC version check against the global table. If the read version is stale, the reader is also marked as *dirty*, and the committed version recorded in the global table becomes its read source during retry. After processing the read set, FaaSRep appends the function's write set to the corresponding `writers` fields. The structure of the dependency graph is shown in Figure 6b.

Besides cross-request dependencies, functions within the same request may form Read-Your-Writes (RYW) relationships [13, 48]. FaaSRep records this RYW information in workflow metadata during initial execution. After validation, these intra-workflow dependencies refine the dependency graph by overriding inter-request dependencies on the same data and propagating dirty marks along RYW edges.

For each of the n data accesses within a workflow, this dependency construction method only adds two extra constant-time operations: checking for the latest writer and updating the writers list. Consequently, the total overhead for building the dependency graph is $O(n)$, including the merge of RYW edges. This maintains the same asymptotic complexity as the standard OCC process [12].

5.2 Hybrid Reconciliation and Commit

After validation constructs the dependency graph, FaaSRep reconciles each batch with a hybrid optimistic/pessimistic protocol, as shown in Algorithm 1. To avoid the problem discussed in Section 3.4.2, FaaSRep uses a *readiness condition* to determine when a request can safely rely on its dependency graph. Intuitively, once this condition holds, the predecessors represented in the request's dependency graph will not later abort and invalidate the graph. The exact definition of this condition is given in Section 5.3.

The common case follows the optimistic path. For each validated batch B , FaaSRep uses the batch version V_B and the equivalent serial order $\prec_B$ produced by validation. Each request $R \in B$ starts in optimistic mode and retries according to the dependency graph G_R constructed under $\prec_B$ (Lines 1–2). However, R cannot commit immediately after retry. It becomes ready to commit only after its retry completes and its readiness condition holds (Line 12).

If a predecessor of R aborts before R satisfies the readiness condition, the optimistic result of R may have been computed under an invalid dependency graph. FaaSRep then switches R to the pessimistic path and discards the optimistic retry result (Lines 7–8). Once R becomes ready, FaaSRep rebuilds G_R by excluding aborted predecessors and retries R under the rebuilt graph (Lines 9–10). Since this graph is rebuilt only after readiness holds, its represented predecessors will not later trigger application-level aborts and invalidate it. Therefore, pessimistic retry is performed at most once.

A batch becomes ready to commit when every request in it has reached a fixed outcome: aborted or ready to commit. FaaSRep does not publish a request as soon as it becomes ready. Instead, it commits the whole batch after all preceding batches have committed (Line 13), preserving the global serial order across batches and preventing a later batch from becoming visible before an earlier one.

During commit, FaaSRep merges the write sets of all non-aborted requests in B according to $\prec_B$ (Line 14). If multiple requests in the batch write the same key, the merged write set W_B keeps only the latest write under $\prec_B$. This is equivalent

Algorithm 1: Hybrid reconciliation protocol

Input: Validated batch B , batch version V_B ,
equivalent serial order $\prec_B$, dependency graph
 G_R for each request $R \in B$
Output: Batch B commits or aborts consistently with
 $\prec_B$

```

1 foreach  $R \in B$  do
2    $\text{mode}(R) \leftarrow \text{optimistic}; \text{terminal}(R) \leftarrow \text{false};$ 
    $\text{readyToCommit}(R) \leftarrow \text{false}; \text{Retry}(R, G_R);$ 
3 while  $\exists R \in B : \neg \text{terminal}(R)$  do
4   foreach  $R \in B$  with  $\neg \text{terminal}(R)$  do
5     if  $\text{ApplicatonAbort}(R)$  then
6        $\text{DiscardWrites}(R); \text{RemoveWriters}(R);$ 
        $\text{terminal}(R) \leftarrow \text{true};$ 
7     else if  $\text{mode}(R) = \text{optimistic and}$ 
        $\text{PredAbortBeforeReady}(R)$  then
8        $\text{DiscardRetryResult}(R);$ 
        $\text{mode}(R) \leftarrow \text{pessimisticWait};$ 
9     else if  $\text{mode}(R) = \text{pessimisticWait and}$ 
        $\text{Ready}(R)$  then
10       $G_R \leftarrow \text{RebuildGraph}(R);$ 
       $\text{mode}(R) \leftarrow \text{pessimisticRetry};$ 
       $\text{Retry}(R, G_R);$ 
11    else if  $\text{Ready}(R) \text{ and } \text{RetryDone}(R)$  then
12       $\text{readyToCommit}(R) \leftarrow \text{true};$ 
13  if  $\text{BatchReady}(B) \text{ and}$ 
     $\text{PrecedingBatchesCommitted}(B)$  then
14     $W_B \leftarrow \text{MergeLatestWrites}(B, \prec_B);$ 
     $\text{PublishToGlobalStorage}(W_B, V_B);$ 
     $\text{FlushCache}(W_B, V_B);$ 
     $\text{UpdateGlobalTable}(W_B, V_B);$ 
     $\text{RemoveBatchWriters}(B);$ 
15    foreach  $R \in B$  with  $\text{readyToCommit}(R)$  do
16       $\text{terminal}(R) \leftarrow \text{true};$ 
```

to executing the requests in $\prec_B$ order and publishing the final value of each key after the batch. FaaSRep then writes W_B to global storage with version V_B , refreshes the cache with the same version, and updates the global table used by later validation (Lines 13–14). Both commit and abort clear the corresponding writers entries (Lines 6 and 14), preventing terminal requests from appearing in future dependencies.

5.3 Readiness Condition

FaaSRep tries to determine as early as possible when a request can safely rely on its dependency graph. The condition should satisfy two requirements. First, it must ensure that the graph will not be invalidated by later application-level aborts. Second, it should be easy to track and should make graph reconstruction cost bounded in the pessimistic path.

FaaSRep achieves this by defining readiness with one batch-level boundary and one request-level boundary.

The batch-level boundary limits cross-batch dependencies. For a request R in batch B , $PBD(R)$ (pessimistic batch dependency) denotes the latest earlier batch that contains an upstream dependency of R . Instead of tracking every individual predecessor outside B , FaaSRep waits until $PBD(R)$ has committed. Since batches commit in order, all earlier batches have also committed at that point. Therefore, cross-batch predecessors will not appear in a rebuilt dependency graph.

The request-level boundary limits dependencies inside the current batch. Let $PTD(R)$ (pessimistic transaction dependency) denote the latest upstream dependency request of R in the equivalent serial order $\prec_B$. FaaSRep waits until every request up to $PTD(R)$ in $\prec_B$ is ready to commit or has become terminal, forming a stable set of in-batch candidates for rebuilding G_R .

Formally, following the definition in Algorithm 1, R satisfies $\text{Ready}(R)$ when both conditions hold:

$PBD(R)$ has committed, and

$\forall T' \preceq_B PTD(R), \text{readyToCommit}(R') \vee \text{terminal}(R')$.

This definition bounds dependency reconstruction. When R enters the pessimistic path, FaaSRep waits for $\text{Ready}(R)$ and rebuilds G_R using only earlier requests in the same batch up to $PTD(R)$. Earlier batches are represented by committed versions in the global table, and later in-batch requests cannot precede R under $\prec_B$. Thus, readiness prevents the rebuilt graph from being invalidated again while avoiding unnecessary serialization of independent predecessors.

5.4 Correctness Analysis

We provide a proof sketch demonstrating that FaaSRep guarantees strict serializability for committed workflow requests. The full proof is detailed in Appendix A. Aborted requests do not contribute any committed writes and are therefore excluded from the committed history.

For each batch B , the validator fixes a request order $\prec_B$. Let S denote the serial order obtained by concatenating batches in their commit sequence. Within each batch, committed requests follow $\prec_B$. The proof relies on three invariants:

Invariant 1: Fixed serial order. For each validated batch B , the order $\prec_B$ is maintained. Re-execution or discarded requests do not alter the relative order of committed requests, ensuring a well-defined serial order S .

Invariant 2: Predecessor stability. Once a request R satisfies $\text{Ready}(R)$, its dependency graph cannot be invalidated by later application-level aborts. Cross-batch predecessors are captured for by $PBD(R)$, and in-batch predecessors by $PTD(R)$.

Invariant 3: Read-source preservation. Every read performed by a committed request R observes the latest preceding write in S . This source is either the upstream writer in

the dependency graph, or a committed version in the global table.

Theorem. FaaSRep ensures strict serializability.

Proof sketch. Invariant 1 provides a fixed candidate serial order for all committed requests. Invariant 2 ensures that requests commit only after predecessors affecting their reads have stabilized, preventing application-level aborts from invalidating the dependency graph. Invariant 3 guarantees that all reads observe the correct source. Therefore, the committed history is equivalent to the serial execution S . Since batches commit in order and validation for a new batch occurs only after previous batches are visible, real-time order is preserved. Hence, FaaSRep provides strict serializability.

6 Runtime Design of FaaSRep

Section 5.2 defines the reconciliation protocol at an abstract level. This section describes how the runtime executes this protocol efficiently.

6.1 Runtime Coordination of Hybrid Reconciliation

The protocol defines several state transitions during reconciliation, including the reconciliation mode and readiness condition of each request. FaaSRep relies on the workflow sink to maintain these states during execution. The sink that submits a validation batch tracks the unfinished requests in the batch and coordinates their progress with the validator.

When retry starts, the validator returns the dependency graph to the control plane and registers it with the corresponding sink. The sink then use it to update the successor information of unfinished requests. These successor relations allow the sink to locate the requests whose mode must change when a predecessor aborts, and to propagate the corresponding state update to them.

The sink also keeps a compact record for each unfinished request, recording its completion status and which mode currently governs it. Separately, the sink evaluates readiness for waiting requests from the completion states of related PBD and PTD dependencies. For requests in pessimistic mode, once readiness holds, the sink notifies the validator to reconstruct the dependency graph and start the pessimistic retry. When all requests in the batch are ready to commit, the sink asks the validator to commit the batch, keeping validation, retry, and commit progress consistent.

6.2 Retry at Function Granularity

FaaSRep retries a request at function granularity to match the serverless execution model [34], as shown in Figure 7. Before retrying a function, the control plane checks its `dirty` flag in the dependency graph. A clean function is skipped and reuses its initial result. A dirty function is re-executed after the control plane prepares its dependencies using the corresponding `DependencyInfo` described in Section 5.1.

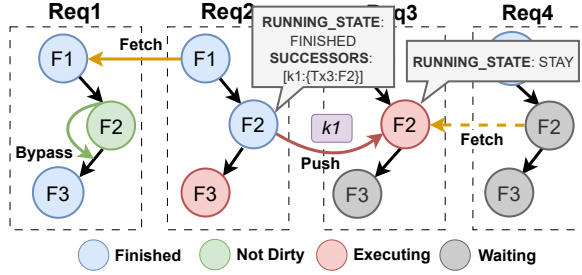


Figure 7. Function-level re-execution procedure. Req1.F2 is clean and is therefore skipped. Req3.F2 pushes the required data to its downstream successor, while Req4.F2 waits until Req3.F2 finishes before re-executing.

DependencyInfo records the read sources required by each dirty function. Before re-execution, the control plane uses this metadata to fetch the required data versions from the cache on the corresponding worker nodes or from global storage. The function still invokes normal data APIs during re-execution, while the sidecar intercepts these calls and serves reads from the prefetched data. In this way, the user function observes the correct version transparently.

A dirty function may start before its upstream functions have produced new outputs. FaaSRep therefore maintains `RUNNING_STATE` and `SUCCESSORS` for each function. `RUNNING_STATE` records whether the function has produced an output consumable by successors, and `SUCCESSORS` records downstream functions waiting for that output. If an upstream function is not ready, the current function registers itself in the upstream function’s `SUCCESSORS` and waits. When the upstream function finishes, it updates `RUNNING_STATE` and pushes the required data to the waiting successors. A skipped clean function performs the same notification because its reused output may also unblock downstream functions.

6.3 Fast-Path for Function Re-execution

In the standard serverless workflow model, each function invocation still passes through the control plane before reaching its target container. During reconciliation, this path can offset the gain from fine-grained retry: even if FaaSRep re-executes only a part of functions, each re-execution may still incur scheduling and forwarding overhead [38].

FaaSRep therefore introduces a **fast path** for the retry phase, as illustrated in Figure 8. After the initial execution, the control plane keeps the used containers in a reserved pool to prevent them from being reclaimed [39]. It also collects the network endpoints of their sidecars and attaches this placement information to the metadata submitted for validation. After validation, the validator combines the placement information with the dependency graph and installs the graph into the cache of the corresponding worker nodes.

During retry, a sidecar reads the locally cached dependency graph to identify upstream functions, required data

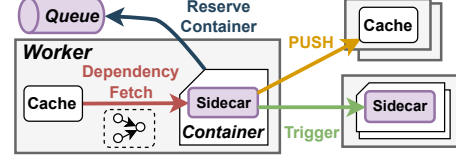


Figure 8. Fast path for reconciliation. During retry, sidecars fetch the dependency graph, trigger successor functions, and push required data to downstream functions.

items, and target sidecar endpoints. It then directly fetches data from upstream nodes, receives outputs pushed by completed predecessors, or wakes downstream containers through successor relations. This container-to-container path avoids repeated control-plane mediation on the critical path and reduces each communication step to a direct transfer between the involved nodes. Large-scale deployments can further benefit from low-latency networking such as RDMA [28, 32].

7 Implementation

FaaSRep is implemented on top of FaaSFlow [35], whose distributed workflow manager reduces cross-node traffic. We extend FaaSFlow’s workflow state to include each function’s read/write sets and the network endpoints of its containers.

Our implementation uses two communication paths. The initial run follows the normal invocation path, where the control plane triggers functions through HTTP REST APIs. In contrast, the “fast-path” for the retry phase is implemented as a peer-to-peer network using gRPC [55]. To ensure security, the fast-path gRPC strictly limits communication to containers within the same workflow. Functions run in an isolated Docker [4] container, with a sidecar packed in the image. This sidecar acts as both a gRPC server and client.

Each worker node hosts a Redis instance, partitioned to serve as a local cache for data, function states and buffered writes. Functions access this local Redis [7] instance and global database through their sidecar. The checking and updating of `RUNNING_STATE` and `SUCCESSORS` are implemented using Redis atomic operations to ensure correctness. The validator is implemented as a scalable event [5] WSGI server combined with a process pool to handle concurrent requests.

The cache maintains multiple versions only on the retry path. Normal execution reads the latest entry, while retry fetches the version specified in the dependency graph from global storage or the upstream worker node. Pinned versions are released after all dependent dirty functions complete.

7.1 Scalable Validator

FaaSRep decouples the validator to prevent it from becoming a scalability bottleneck in high-concurrency environments. Based on real-world data [45] and insights from previous work [50], we first observed that serverless applications exhibit high data isolation, with 99.7% of data not being shared

across multiple applications. Consequently, FaaSRep constructs validators at the application granularity, providing each application with its own dedicated validator and workflow sink instance. This model can seamlessly scale up or down as the number of applications changes.

However, even a dedicated validator is still limited by its single-instance nature. To address this, FaaSRep further decomposes each validator into a single-instance *serializer* and multiple scalable *processors*. A proxy distributes incoming batch validation requests across the processors.

Serializer maintains the global table mentioned in Section 5.1, sequentially checks versions and finds dependencies. Multiple processors concurrently receive batches and prepare preliminary data, interacting with the serializer via a work queue. After receiving results from the serializer, they proceed with subsequent operations such as constructing the graph and sending retry requests. In the end, the serializer follows the order of writers to commit batches.

7.2 Failure Recovery

For the storage layer, FaaSRep relies on the underlying database’s own fault tolerance mechanisms [47]. Here, we focus on failures of compute nodes and the FaaSRep validator.

At runtime, FaaSRep continuously monitors the status of each node. If a crash is detected, the validator discards all uncommitted requests, including those currently being retried or awaiting validation, and retries them after recovery. Requests that have already been committed are unaffected, as their modifications have been fully recorded.

Some batches may be interrupted during the commit phase, resulting in partially committed data. To address this, FaaSRep logs every batch’s modifications in the database before committing, denoted as a *delta set*. Fully completed commit records its completion log. During recovery, any batch without this log recorded must roll back its changes.

When a validator crashes, all validation and retry operations of the corresponding application are paused. During recovery, the validator contacts the gateway to identify any uncommitted requests and retrieves their corresponding delta set logs. After rolling back modifications, the validator reconstructs its global table.

7.3 Handling Dynamic Transactions

The scope in Section 4 covers common transactional workflows, such as transfers [62] that access the source and destination accounts, and orders [9, 10] that access the corresponding inventory and order records. In these cases, the access set is discovered during execution, and remains stable across retries of the same request.

Some workflows may violate this property. For example, a function may access an object randomly, causing a retry to access a different object from the initial execution. FaaSRep handles such cases conservatively: it aborts the current

retry, sends the request to a new round, and degrades affected successors to pessimistic mode. This preserves correctness because the pessimistic mode of FaaSRep provides the safety fallback, but may reduce performance. If dynamic accesses occur frequently, FaaSRep may behave close to the pessimistic lower bound in Section 8.6.

8 Evaluation

This section evaluates FaaSRep across different workloads, contention levels, and deployment configurations. We first compare FaaSRep with state-of-the-art transactional serverless systems to measure its end-to-end performance. We then isolate each major mechanism, including workflow reconciliation, function-level retry, and fast-path communication, to quantify its contribution. Finally, we examine the system overhead and scalability of FaaSRep.

8.1 Experimental Setup

All experiments run on an 8-node cluster with one storage node and seven worker nodes. Each node has a 32-core Intel Xeon Platinum 8562Y+ CPU and 256 GB memory. Docker containers are capped at 1 CPU core and 256 MB memory. The storage node hosts the gateway and FaaSRep validator. We use ScyllaDB [8] as the global storage backend for data, metadata, and logs, with strong-consistency reads enabled.

We compare FaaSRep with two strict-serializable baselines. Beldi [59] uses remote locking with Wait-Die deadlock prevention. Concord [50] uses the cache coherence protocol. Both Beldi and Concord allow concurrent reads, similar to read-write locks. We also implement an OCC variant that represents FaaSRep without reconciliation. For fairness, all systems share the same FaaSFlow [35] codebase.

Our evaluation uses two workload classes. The microbenchmark includes chain workflows with lengths 2, 4, 8, and 16 (*c2*, *c4*, *c8*, and *c16*), and fan-out workflows with degrees 2, 4, 6, and 8 (*w2*, *w4*, *w6*, and *w8*). These workflows access 4 KB objects from 10,000 keys following a Zipf distribution [17]. The real-world applications are from prior work [3, 30, 59, 62]: **Travel reservation (Travel)** reserves flights and cars before confirming payment, and may abort when the resource is insufficient. **Banking system (Bank)** transfers money between two accounts. **Social media (Social)** lets users comment on posts and publish timeline updates atomically.

We use closed-loop microbenchmarks to analyze FaaSRep across workflow shapes and access patterns, then evaluate real-world applications under both closed-loop execution and open-loop execution driven by Azure traces [45, 61]. Since these traces lack per-request data-access information, they capture realistic arrival patterns rather than real-world conflict distributions. Our microbenchmarks instead cover a broad range of conflict intensities in Section 8.3.

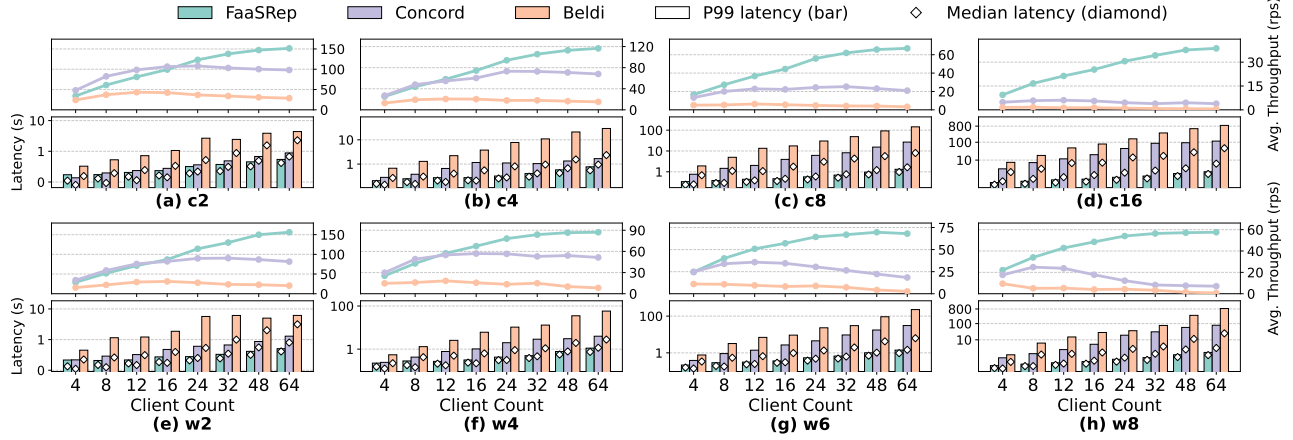


Figure 9. P99/P50 latency and average throughput for workflows with different lengths and fan-out degrees.

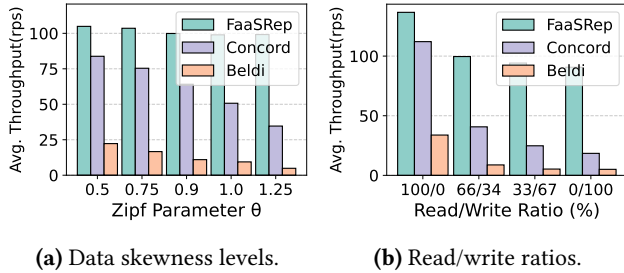


Figure 10. Performance under varying data contention.

8.2 Performance Across Workflow Structures

This experiment evaluates FaaSRep across different application styles using our micro-benchmark. We set the Zipf factor to 0.9, following mainstream configurations [23, 43, 52], and use a read/write ratio of 2:1 to model a hybrid pattern.

Figure 9 shows that FaaSRep outperforms both baselines in most settings. With 32 clients, FaaSRep achieves 1.33–8.81× (4.06–41.35×) higher throughput than Concord (Beldi). Its median latency is sometimes close to Concord, since both systems benefit from the cache. The main difference lies in tail latency: FaaSRep reduces P99 latency by more than 23.58% over Concord and 84.72% over Beldi, because reconciliation avoids repeated retry cycles after conflicts.

The performance gap becomes larger as the client count or workflow scale increases. Longer chains waste more completed functions when a request is aborted, and high fan-out workflows create more concurrent accesses to shared objects. As a result, Beldi and Concord degrade in both throughput and latency. In the extreme case of *c16* with 64 clients, both baselines deliver less than 4 rps, while FaaSRep remains effective across all tested structures. Beldi performs worst across settings, mainly because its remote-locking mechanism adds communication overhead to each data access.

FaaSRep also shows a trade-off in simple low-contention cases. For example, in the *c2* workflow with 4 clients, its throughput is 0.71× that of Concord. In such a lightly loaded setting, conflicts are rare, and the fixed cost of validation and reconciliation is harder to amortize. This scenario therefore favors a simpler protocol than FaaSRep.

8.3 Performance Under Different Data Contention

This experiment studies how FaaSRep behaves under different levels of data contention. We vary two factors: the Zipf factor θ from 0.5 to 1.25 to control data skewness, and the read/write ratio from read-only to write-only. We report the conflict-affected rate, defined as the fraction of requests in the baselines that require more than one execution round because of conflicts. In this section and all subsequent closed-loop tests, we use *c4* with 32 clients as a moderate workload.

Figure 10a shows the impact of data skewness. As θ increases, accesses concentrate on fewer hot keys, causing the baselines to suffer more failed attempts. The conflict-affected rate rises from 9.06% to 64.72% in Concord and from 10.91% to 76.41% in Beldi. Concord remains competitive under low skew because of cached reads, but its throughput drops quickly when hot keys dominate the workload. In contrast, FaaSRep keeps its throughput stable, decreasing only from 104.9 to 99.14 rps. The reason is that the retry cost of FaaSRep is determined by how many functions become dirty. Retrying a dirty function incurs similar overhead regardless of whether all or only part of its data accesses conflict.

Figure 10b shows the impact of read/write ratio. Read-only scenes are contention-free, allowing transactions to execute in parallel. When writes are introduced, the conflict-affected rate rises sharply. For Concord, it reaches 54.59% and 61.81% when the read ratio is 0.66 and 0.33, respectively. For Beldi, it reaches 64.69% and 75.48%. FaaSRep still performs best in the read-only case, because its batch validation and commit amortize communication overhead.

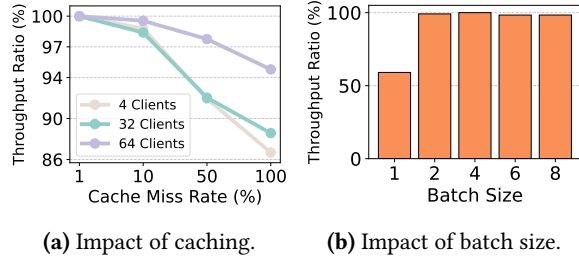


Figure 11. Effects of different caching and batching settings.

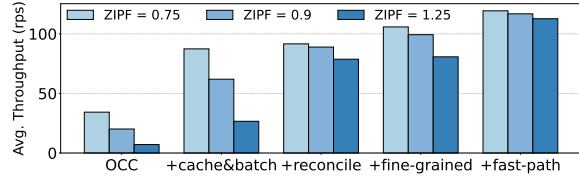


Figure 12. Design breakdown of FaaSRep.

8.4 Sensitivity Analysis

FaaSRep is derived from OCC and also uses caching and batching to improve performance. In this section, we change configurations of these two methods to evaluate their impact.

First, we control the cache miss rate for each data access, and calculate the throughput degradation compared to the case with 1% miss rate. Figure 11a shows the results. Throughput drops as the miss rate increases, but less than 13.3% in all settings. It indicates that FaaSRep’s performance improvement is not mainly from caching, but from its effective reconciliation mechanism.

For batching, we change the batch size from 1 to 8, and measure the throughput ratio compared to the best case. Figure 11b shows that the main improvement occurs when the batch size increases to 2, with an increase of 67.9%. The subsequent growth shows a diminishing marginal effect, as larger batch is harder to fill, leading to untimely validation.

8.5 Ablation Study

Figure 12 illustrates the impact of FaaSRep’s key design components under varying contention levels. When no additional mechanisms are applied, FaaSRep degrades to OCC. In this setup, enabling caching and batching can improve the performance by 154.8-274.3%. The reconciliation mechanism yields further throughput gains ranging from 4.83% to 195.63%. This benefit becomes especially pronounced as contention intensifies, which can be properly addressed by FaaSRep.

Moreover, enabling fine-grained retry (§6.2) contributes an additional 2.52-15.51% improvement. At this time, low-contention scenarios benefit more, where a higher proportion of functions remain unaffected by conflicts and can be skipped during the retry phase, thereby further reducing unnecessary computation.

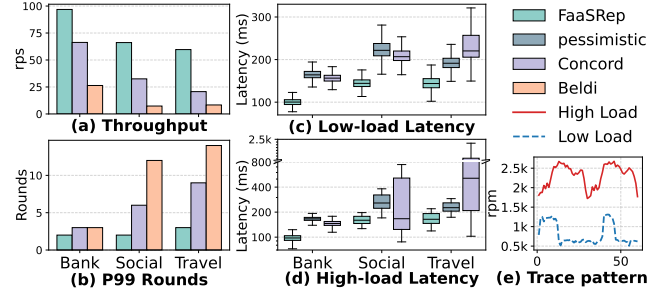


Figure 13. Real-world application performance under closed-loop and trace-driven workloads.

Finally, when the fast-path optimization (§6.3) is enabled, the throughput increases by another 12.7-39.6%, showing the effectiveness of bypassing the control plane. The benefit is also more noticeable under high contention settings, which introduce heavier loads to the control plane.

8.6 Performance in Real-world Scenarios

We evaluate FaaSRep with the three real-world applications described in Section 8.1. This experiment includes a closed-loop test for throughput and an open-loop test driven by production-level traces. Some requests may proactively trigger aborts when application conditions are not satisfied, such as insufficient balance or inventory.

Figure 13(a) reports results in the closed-loop setting. FaaSRep achieves the highest throughput across all applications, improving throughput by 3.67-9.01 $\times$ over Beldi and 1.46-2.73 $\times$ over Concord. The improvement is smaller for Bank because it is simpler and its conflicts are less frequent. This trend is also reflected by the P99 execution rounds in Figure 13(b). While the baselines may require many rounds to resolve conflicts, FaaSRep keeps it bounded. Even when a request falls back to pessimistic reconciliation after an application-level abort, it finishes within at most three rounds.

Figure 13(c) and (d) further evaluate FaaSRep on two 1-hour traces from Azure [45, 61]. Since no real application-level pattern is available in these traces, we also include *pessimistic* as a conservative lower bound. This variant forces requests to fall back to pessimistic reconciliation, allowing us to study the worst-case impact when fallback happens frequently. In the normal mode, the fraction of application-aborted requests ranges from 3.24% to 6.67% across the three applications. FaaSRep still outperforms Concord in all tested cases, showing that the hybrid design handles application-level aborts without imposing conservative execution on the common case. The pessimistic variant increases latency under low load because requests wait on dependencies, but its latency remains close to Concord. Under high load, Concord suffers sharp tail-latency growth from repeated retries, while both FaaSRep variants keep latency bounded.

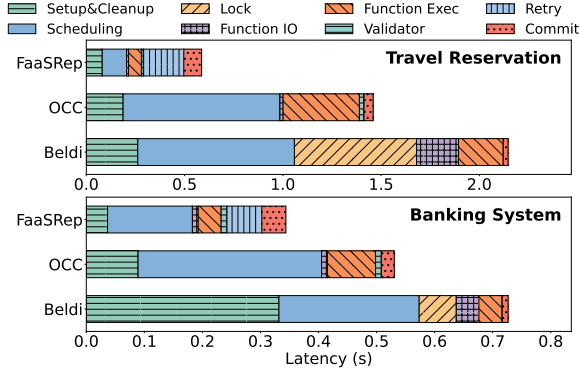


Figure 14. Execution time breakdown in scenarios with high/low-contention.

8.7 Execution Time Breakdown

This experiment breaks down the average latency in the closed-loop setting to explain where the performance gap comes from. We use Travel as a high-contention workload and Banking as a lower-contention workload.

As shown in the top half of Figure 14, although OCC leverages cache to achieve a lower function IO latency than Beldi, both of them suffer from high scheduling overhead and function execution time caused by the contention-based retries. Beldi faces an additional penalty from its remote lock mechanism, surpassing the actual function execution time. Furthermore, a notable portion of Beldi’s latency is consumed by the cleanup of lock entries. In contrast, while FaaSRep introduces specific overheads for reconciliation and sequential commit, these costs are significantly outweighed by the savings achieved from eliminating repeated re-runs.

In the low-contention scenario, OCC and Beldi improve because fewer attempts are wasted on conflicts. FaaSRep still achieves lower latency because of less reconciliation work. But its protocol overhead, such as sequential commit, becomes more visible. This result also clarifies the boundary of FaaSRep’s benefit. When workflows are extremely simple and conflicts are rare, the fixed cost of reconciliation is harder to amortize, so the performance gap can narrow.

8.8 Overhead and Scalability

This experiment first evaluates whether FaaSRep introduces substantial resource overhead. We compare FaaSRep to OCC, with the three real-world applications co-located. We use CPU usage rather than CPU time as the CPU metric. The validator is deployed on the storage backend, so it is excluded from this part and evaluated separately below. As shown in Figure 15, FaaSRep increases module CPU usage by only 4.1% and module memory usage by 0.5%. The sidecar increases container memory usage by 4.2%. Although bandwidth usage increases by 41.5%, the absolute increase is only 1278.83 KB/s, mainly from fast-path communication and metadata

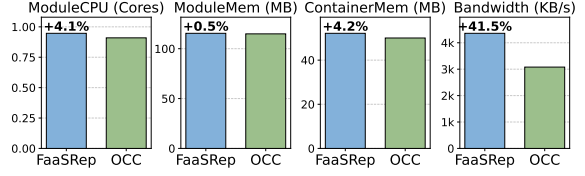


Figure 15. Display of FaaSRep overhead.

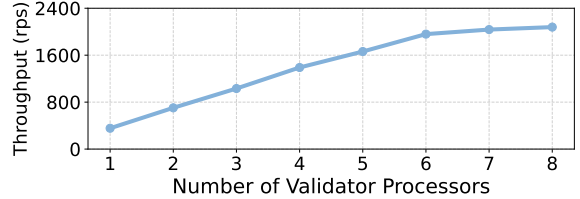


Figure 16. The processing capability of FaaSRep validator with different numbers of workers.

exchange during retry. The additional memory inside function containers is negligible, so the user-visible monetary cost does not increase in practice.

We next evaluate scalability. FaaSRep uses the same distributed function execution framework as prior serverless systems [35, 58, 59], so the main new scalability concern is the validator. The validator includes a sequential stage for validation and commit, but most preprocessing and metadata generation can be parallelized across processors. We therefore measure the maximum validation throughput while varying the number of validator processors.

Figure 16 shows that the validator throughput increases with the number of processors and saturates at eight processors, reaching 2078.47 rps. This capacity is higher than the peak request rate of many real-world serverless traces, such as the Azure Function trace [45], whose peak is no more than 500 rps. It is also above the default concurrency limit of common commercial platforms, such as the 1000 concurrent-function limit per AWS Lambda account [2].

9 Conclusion

In this paper, we present FaaSRep, a high-performance transactional system for serverless workflows with strict serializability. It uses workflow reconciliation to turn conflicts discovered in an initial attempt into a scheduled retry constrained by function-level dependencies. Specifically, it delays dependency construction at validation time to avoid runtime overhead, and proposes fine-grained function-level retry with a peer-to-peer fast-path to minimize control plane involvement. Moreover, it introduces a hybrid reconciliation protocol to correctly handle application-level aborts of the workflow requests. Our evaluation demonstrates that FaaSRep significantly improves system throughput and reduces the response latency compared to existing solutions.

References

- [1] 2025. AWS example for serverless transactional workflows. <https://aws.amazon.com/cn/blogs/compute/building-a-serverless-distributed-application-using-a-saga-orchestration-pattern/>.
- [2] 2025. AWS Lambda quotas: limits related to AWS Lambda. <https://docs.amazonaws.cn/lambda/latest/dg/gettingstarted-limits.html>.
- [3] 2025. DeathStarBench: A Benchmark Suite for Microservices. <https://github.com/delimitrou/DeathStarBench/>.
- [4] 2025. Docker: Accelerated Container Application Development. <https://www.docker.com/>.
- [5] 2025. Gevent: A coroutine-based Python networking library. <https://www.gevent.org/>.
- [6] 2025. Microservices-Based Banking Systems: Enabling Innovation Through Architectural Independence. <https://bicon.li/2025/05/01/microservices-based-banking-systems/>.
- [7] 2025. Redis: In-memory data structure store, used as a database, cache, and message broker. <https://redis.io/>.
- [8] 2025. ScyllaDB | Monstrously Fast + Scalable NoSQL. <https://www.scylladb.com/>.
- [9] 2025. Serverless App Engine of Aliyun. <https://www.aliyun.com/product/aliware/sae>.
- [10] 2025. tySupermarket for worry-free purchasing. <https://ztf58.cn/>.
- [11] Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. 2019. A survey on scheduling strategies for workflows in cloud environment and emerging trends. *ACM Computing Surveys (CSUR)* 52, 4 (2019), 1–36.
- [12] Marcos K Aguilera, Arif Merchant, Mehul Shah, Alistair Veitch, and Christos Karamanolis. 2007. Sinfonia: a new paradigm for building scalable distributed systems. *ACM SIGOPS Operating Systems Review* 41, 6 (2007), 159–174.
- [13] Hesam Nejati Sharif Aldin, Hossein Deldari, Mohammad Hossein Moattar, and Mostafa Razavi Ghods. 2020. Strict timed causal consistency as a hybrid consistency model in the cloud environment. *Future Generation Computer Systems* 105 (2020), 259–274.
- [14] Ahsan Ali, Riccardo Pincirol, Feng Yan, and Evgenia Smirni. 2020. Batch: Machine learning inference serving on serverless platforms with adaptive batching. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–15.
- [15] Sadjad Fouladi and. 2019. From Laptop to Lambda: Outsourcing Everyday Jobs to Thousands of Transient. In *Proceedings of the 2019 USENIX Annual Technical Conference, USENIX, Dahlia Malkhi and (Ed.)*. USENIX Association, 475–488.
- [16] Lixiang Ao, Liz Izhikevich, Geoffrey M. Voelker, and George Porter. 2018. Sprocket: A Serverless Video Processing Framework. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC 2018, Carlsbad, CA, USA, October 11-13, 2018*. ACM, 263–274.
- [17] Robert L Axtell. 2001. Zipf distribution of US firm sizes. *science* 293, 5536 (2001), 1818–1820.
- [18] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. 1995. A critique of ANSI SQL isolation levels. *ACM SIGMOD Record* 24, 2 (1995), 1–10.
- [19] Philip A. Bernstein, David W. Shipman, and Wing S. Wong. 1979. Formal aspects of serializability in database concurrency control. *IEEE Transactions on Software Engineering* 3 (1979), 203–216.
- [20] João Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy H. Katz. 2019. Cirrus: a Serverless Framework for End-to-end ML Workflows. In *Proceedings of the ACM Symposium on Cloud Computing, SoCC 2019, Santa Cruz, CA, USA, November 20-23, 2019*. ACM, 13–24. doi:10.1145/3357223.3362711
- [21] Benjamin Carver, Jingyuan Zhang, Ao Wang, Ali Anwar, Panruo Wu, and Yue Cheng. [n. d.]. Rodrigo Fonseca, Christina Delimitrou, and Beng Chin Ooi (Eds.).
- [22] Youmin Chen, Xiangyao Yu, Paraschos Koutris, Andrea C Arpaci-Dusseau, Remzi H Arpaci-Dusseau, and Jiwu Shu. 2022. Plor: General transactions with predictable, low tail latency. In *Proceedings of the 2022 International Conference on Management of Data*. 19–33.
- [23] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing*. 143–154.
- [24] Mohammad Dashti, Sachin Basil John, Amir Shaikhha, and Christoph Koch. 2017. Transaction repair for multi-version concurrency control. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 235–250.
- [25] Shlomi Dolev, Amos Israeli, and Shlomo Moran. 1993. Self-stabilization of dynamic systems assuming only read/write atomicity. *Distributed Computing* 7, 1 (1993), 3–16.
- [26] Zhiyuan Dong, Zhaoguo Wang, Xiaodong Zhang, Xian Xu, Changgeng Zhao, Haibo Chen, Aurojit Panda, and Jinyang Li. 2023. Fine-grained re-execution for efficient batched commit of distributed transactions. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1930–1943.
- [27] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the twenty-fourth international conference on architectural support for programming languages and operating systems*. 3–18.
- [28] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. 2016. RDMA over commodity ethernet at scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*. 202–215.
- [29] Zhihan Guo, Kan Wu, Cong Yan, and Xiangyao Yu. 2021. Releasing locks as early as you can: Reducing contention of hotspots by violating two-phase locking. In *Proceedings of the 2021 International Conference on Management of Data*. 658–670.
- [30] Zhipeng Jia and Emmett Witchel. 2021. Boki: Stateful serverless computing with shared logs. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 691–707.
- [31] Eric Jonas, Johann Schleier-Smith, Vikram Sreekanti, Chia-Che Tsai, Anurag Khandelwal, Qifan Pu, Vaishaal Shankar, Joao Carreira, Karl Krauth, Neeraja Yadwadkar, et al. 2019. Cloud programming simplified: A Berkeley view on serverless computing. *arXiv preprint arXiv:1902.03383* (2019).
- [32] Gurkirat Kaur and Manju Bala. 2013. Rdma over converged ethernet: A review. *International Journal of Advances in Engineering & Technology* 6, 4 (2013), 1890.
- [33] Hsiang-Tsung Kung and John T Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems (TODS)* 6, 2 (1981), 213–226.
- [34] Zijun Li, Linsong Guo, Jiagan Cheng, Quan Chen, Bingsheng He, and Minyi Guo. 2022. The serverless computing survey: A technical primer for design architecture. *ACM Computing Surveys (CSUR)* 54, 10s (2022), 1–34.
- [35] Zijun Li, Yushi Liu, Linsong Guo, Quan Chen, Jiagan Cheng, Wenli Zheng, and Minyi Guo. 2022. Faasflow: Enable efficient workflow execution for function-as-a-service. In *Proceedings of the 27th acm international conference on architectural support for programming languages and operating systems*. 782–796.
- [36] Zijun Li, Chuhao Xu, Quan Chen, Jieru Zhao, Chen Chen, and Minyi Guo. 2023. Dataflower: Exploiting the data-flow paradigm for serverless workflow orchestration. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*. 57–72.

- [37] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *European conference on computer vision*. Springer, 740–755.
- [38] Guowei Liu, Laiping Zhao, Yiming Li, Zhaolin Duan, Sheng Chen, Yitao Hu, Zhiyuan Su, and Wenyu Qu. 2024. Fuyao: Dpu-enabled direct data transfer for serverless computing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 431–447.
- [39] Qichang Liu, Yue Cheng, Haiying Shen, Ao Wang, and Bharathan Balaji. 2025. Concurrency-Informed Orchestration for Serverless Functions. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 147–161.
- [40] Taras Lykhenko, Rafael Soares, and Luis Rodrigues. 2021. Faastcc: Efficient transactional causal consistency for serverless computing. In *Proceedings of the 22nd International Middleware Conference*. 159–171.
- [41] Ashraf Mahgoub, Karthick Shankar, Subrata Mitra, Ana Klimovic, Somali Chaterji, and Saurabh Bagchi. 2021. {SONIC}: Application-aware data passing for chained serverless applications. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 285–301.
- [42] Ashraf Mahgoub, Edgardo Barsallo Yi, Karthick Shankar, Sameh Elnikety, Somali Chaterji, and Saurabh Bagchi. 2022. {ORION} and the three rights: Sizing, bundling, and prewarming for serverless {DAGs}. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 303–320.
- [43] Yandong Mao, Eddie Kohler, and Robert Tappan Morris. 2012. Cache craftiness for fast multicore key-value storage. In *Proceedings of the 7th ACM european conference on Computer Systems*. 183–196.
- [44] Sheng Qi, Xuanzhe Liu, and Xin Jin. 2023. Halfmoon: Log-optimal fault-tolerant stateful serverless computing. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 314–330.
- [45] Mohammad Shahradd, Rodrigo Fonseca, Inigo Gouri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX annual technical conference (USENIX ATC 20)*. 205–218.
- [46] Abraham Silberschatz, James L Peterson, and Peter B Galvin. 1991. *Operating system concepts*. Addison-Wesley Longman Publishing Co., Inc.
- [47] Swaminathan Sivasubramanian. 2012. Amazon dynamoDB: a seamlessly scalable non-relational database service. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 729–730.
- [48] Vikram Sreekanti, Chenggang Wu, Saurav Chhatrapati, Joseph E Gonzalez, Joseph M Hellerstein, and Jose M Faleiro. 2020. A fault-tolerance shim for serverless computing. In *Proceedings of the Fifteenth European Conference on Computer Systems*. 1–15.
- [49] Vikram Sreekanti, Chenggang Wu, Xiayue Charles Lin, Johann Schleier-Smith, Jose M Faleiro, Joseph E Gonzalez, Joseph M Hellerstein, and Alexey Tumanov. 2020. Cloudburst: Stateful functions-as-a-service. *arXiv preprint arXiv:2001.04592* (2020).
- [50] Jovan Stojkovic, Chloe Alverti, Alan Andrade, Nikoleta Iliakopoulou, Hubertus Franke, Tianyin Xu, and Josep Torrellas. 2025. Concord: Rethinking Distributed Coherence for Software Caches in Serverless Environments. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 458–473.
- [51] Dixon Tang and Aaron J Elmore. 2018. Toward coordination-free and reconfigurable mixed concurrency control. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*. 809–822.
- [52] Alexander Thomson, Thaddeus Diamond, Shu-Chun Weng, Kun Ren, Philip Shao, and Daniel J Abadi. 2012. Calvin: fast distributed transactions for partitioned database systems. In *Proceedings of the 2012 ACM SIGMOD international conference on management of data*. 1–12.
- [53] Alexandre Verbitski, Anurag Gupta, Debanjan Saha, Murali Brahmasam, Kamal Gupta, Raman Mittal, Sailesh Krishnamurthy, Sandor Maurice, Tengiz Kharatishvili, and Xiaofeng Bao. 2017. Amazon aurora: Design considerations for high throughput cloud-native relational databases. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 1041–1052.
- [54] Tianzheng Wang and Hideaki Kimura. 2016. Mostly-optimistic concurrency control for highly contended dynamic workloads on a thousand cores. *Proceedings of the VLDB Endowment* 10, 2 (2016), 49–60.
- [55] Xingwei Wang, Hong Zhao, and Jiakeng Zhu. 1993. GRPC: A communication cooperation mechanism in distributed systems. *ACM SIGOPS Operating Systems Review* 27, 3 (1993), 75–86.
- [56] Chenggang Wu, Vikram Sreekanti, and Joseph M Hellerstein. 2020. Transactional causal consistency for serverless computing. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 83–97.
- [57] Yingjun Wu, Chee-Yong Chan, and Kian-Lee Tan. 2016. Transaction healing: Scaling optimistic concurrency control on multicores. In *Proceedings of the 2016 International Conference on Management of Data*. 1689–1704.
- [58] Chuhao Xu, Yiyu Liu, Zijun Li, Quan Chen, Han Zhao, Deze Zeng, Qian Peng, Xueqi Wu, Haifeng Zhao, Senbo Fu, et al. 2024. Faasmem: Improving memory efficiency of serverless computing with memory pool architecture. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 331–348.
- [59] Haoran Zhang, Adney Cardoza, Peter Baile Chen, Sebastian Angel, and Vincent Liu. 2020. Fault-tolerant and transactional stateful serverless workflows. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. 1187–1204.
- [60] Haoran Zhang, Shuai Mu, Sebastian Angel, and Vincent Liu. 2025. Causalmesh: A causal cache for stateful serverless computing. *ACM SIGMOD Record* 54, 1 (2025), 48–57.
- [61] Yanqi Zhang, Inigo Gouri, Gohar Ifran Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and cheaper serverless computing on harvested resources. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*. 724–739.
- [62] Jianjun Zhao, Haikun Liu, Shuhao Zhang, Haodi Lu, Yancan Mao, Zhuohui Duan, Xiaofei Liao, and Hai Jin. 2025. Towards {High-Performance} Transactional Stateful Serverless Workflows with {Affinity-Aware} Leasing. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. 1535–1551.

A Proof of Strict Serializability

This appendix proves that FaaSRep guarantees strict serializability for committed workflow requests. The proof considers only states accessed through FaaSRep’s transactional data APIs. We assume that functions inside the transactional boundary are deterministic: given the same request input and the same values returned by transactional reads, they produce the same control flow, read set, and write set. A request that proactively aborts discards all buffered writes and does not enter the committed history. Requests that do not satisfy this scope are handled as discussed in Section 7.3. **Committed history.** Let H denote the history produced by FaaSRep. A workflow request R commits only when FaaSRep publishes its buffered writes to global storage. A request that aborts before commit produces no committed writes and is

removed from H . The committed history H_c is the projection of H that contains only committed requests and the reads and writes performed by those requests.

Batch order and serial order. FaaSRep validates requests in batches. For each batch B , the validator fixes a total request order $\prec_B$ according to the order in which requests are added to B . This order is never changed by retry, fallback, or commit. Batches commit sequentially. Let $B_1, B_2, \dots$ be the batch commit sequence. We define the candidate serial order S by concatenating the committed requests in each committed batch according to $\prec_B$. Aborted requests are skipped.

Dependency graph. For a request R , FaaSRep constructs a dependency graph G_R at validation time. The graph contains intra-request and inter-request edges. An intra-request edge captures a read-your-writes dependency inside R : if a function in R reads a value written earlier by another function in the same request, the reader depends on the writer. An inter-request edge captures a conflicting access between requests: if a read in R should observe a write from an earlier request under $\prec_B$ and the batch order, the reader depends on the corresponding writer. If a read observes an outdated committed version, the reading function is marked dirty and is re-executed. Dirty status is propagated along workflow dependencies, so any function whose input may change is also re-executed. Clean functions reuse their previous results.

Read source. For a read r of key x in a committed request R , its serial source in S is the latest write to x that precedes r in S . This source may be a previous write inside R , a write by an earlier committed request in the same batch, a write by an earlier committed batch, or the initial committed version if no preceding write exists.

Readiness. FaaSRep uses $\text{Ready}(R)$ to decide whether the dependency graph used by R is stable. Let $\text{PBD}(R)$ denote the latest previous batch that contains a cross-batch predecessor of R . If no such batch exists, this condition is treated as satisfied. Let $\text{PTD}(R)$ denote the latest predecessor of R inside the current batch according to $\prec_B$. If no such predecessor exists, this condition is also treated as satisfied. A request R satisfies $\text{Ready}(R)$ when every batch up to $\text{PBD}(R)$ has committed and every request up to $\text{PTD}(R)$ in the current batch has reached a resolved state. A resolved request has either aborted and discarded its writes, or finished retry and become ready to commit. Therefore, once $\text{Ready}(R)$ holds, the outcomes of all predecessors that may affect R 's reads are fixed.

Invariant 1: Fixed serial order. For every committed batch B , the relative order of committed requests in B is exactly $\prec_B$. Therefore, the serial order S is well defined.

Proof. The validator assigns $\prec_B$ once during validation according to the order in which requests are added to B . FaaSRep may re-execute a request, discard a retry result, switch the request from optimistic mode to pessimistic mode, or remove an aborted request, but none of these operations

changes the relative order assigned by the validator. During commit, the validator merges writes in the order of $\prec_B$. Since batches commit sequentially, concatenating the committed subsequences of all batches gives a unique serial order S . This proves the invariant.

Invariant 2: Predecessor stability. If a committed request R commits using a dependency graph G_R , then every predecessor that may affect R 's reads has a fixed outcome before R commits. In particular, no later proactive abort can invalidate G_R .

Proof. FaaSRep allows R to commit only after $\text{Ready}(R)$ holds. By the definition of $\text{Ready}(R)$, all cross-batch predecessors of R belong to batches no later than $\text{PBD}(R)$, and these batches have already committed. Such predecessors cannot later abort or change their writes. The same condition also requires every relevant in-batch predecessor up to $\text{PTD}(R)$ to be resolved. If such a predecessor aborts, its writes are discarded and it is no longer a valid writer. If it finishes retry and becomes ready to commit, it has no remaining execution that can later trigger a proactive abort.

If R remains in optimistic mode, FaaSRep still checks $\text{Ready}(R)$ before allowing it to commit. If a predecessor aborts before this point, $\text{PredAbortBeforeReady}(R)$ forces R to discard its optimistic retry result and switch to pessimistic waiting. If R enters pessimistic mode, FaaSRep waits until $\text{Ready}(R)$ holds, rebuilds G_R after removing aborted predecessors, and re-executes R with the rebuilt graph. Therefore, the graph used by any committed request is constructed or accepted only after all relevant predecessor outcomes have become stable. This proves the invariant.

Invariant 3: Read-source preservation. Every read performed by a committed request R observes its serial source in S .

Proof. Consider any read r of key x in a committed request R . We show that r observes the latest write to x that precedes it in S .

First, suppose the serial source of r is a previous write inside the same request R . FaaSRep records this read-your-writes relation during execution and inserts an intra-request dependency from the writer function to the reader function. During retry, the reader cannot execute using an older external value before the writer has produced the value required by R 's workflow order. Thus, r observes the intra-request source.

Second, suppose the serial source of r is not an intra-request read-your-writes source, but a write by an earlier request in the same batch. During validation, requests are processed according to $\prec_B$. The writer metadata in the global table records uncommitted writers for each key. When the validator checks r , the latest preceding writer of x in the current batch is either present in this metadata or no such

writer exists. If such a writer exists, FaaSRep adds the corresponding inter-request dependency and marks the reader dirty when the original value may be stale. The reader is then re-executed after the writer has produced its value, unless the writer aborts. If the writer aborts, Invariant 2 ensures that R cannot commit using an invalidated graph. It either falls back to pessimistic mode and rebuilds the graph, or waits until the predecessor outcome is fixed. Therefore, when R commits, r observes the latest non-aborted preceding writer in the batch.

Third, suppose the serial source of r is a write from an earlier committed batch. Since batches commit sequentially, all earlier batches are visible before the current batch commits. During validation, FaaSRep compares the version read by r with the committed version in the global table. If the read version is already the latest committed version, the original read is valid. If the read version is stale, the reading function is marked dirty, re-executed, and forced to read the committed version. By Invariant 2, any cross-batch predecessor that may affect this read has committed before R commits. Thus, the re-executed read observes the committed serial source.

Finally, if no preceding write to x exists in S , the serial source is the initial committed version. The same validation rule applies: a read that observes an invalid old version is marked dirty and re-executed, while a read that already matches the global committed version remains valid. In all cases, r observes its serial source in S . This proves the invariant.

Lemma 1: The committed history is equivalent to S .

The committed history H_c is equivalent to the serial execution S .

Proof. By Invariant 1, every committed request has a fixed position in S . By Invariant 3, every read in every committed request observes the same value that it would observe in the serial execution S . Writes are buffered during execution and become visible only when the request commits as part of its batch. Aborted requests discard their buffered writes and are not included in H_c . Therefore, the reads and writes in H_c are identical to those produced by executing the committed requests serially according to S . This proves the lemma.

Lemma 2: The serial order S respects real-time order.

If a committed request R_1 completes before another request R_2 begins, then R_1 precedes R_2 in S .

Proof. A request completes only after its batch has committed and its writes, if any, have been published to global storage. If R_1 completes before R_2 begins, then the batch containing R_1 has already committed before R_2 can be validated. FaaSRep validates a new batch only after previous committed batches are visible to the validator and the global table. Therefore, the batch containing R_2 appears after the batch containing R_1 in the batch commit sequence. Since

S is formed by concatenating batches in commit order, R_1 precedes R_2 in S . This proves the lemma.

Theorem. FaaSRep guarantees strict serializability for committed workflow requests.

Proof. By Lemma 1, the committed history H_c is equivalent to the serial execution S . Thus, H_c is serializable. By Lemma 2, S respects the real-time order of non-overlapping committed requests. Therefore, H_c is strictly serializable. Since aborted requests do not produce committed writes and are excluded from H_c , they do not affect the argument. Hence, FaaSRep guarantees strict serializability for committed workflow requests.